G09 - 项目结题会议纪要

一、 会议基本信息

- **会议主题：** 软件工程管理第9组项目结题会议
- **会议时间：** 2025年12月8日

二、 会议议程与主要内容

议程一：项目后续优化内容确定

1：智能问答中心优化

一：语义分块

- 目前的切分方式（句子窗口）是机械的。它不管这句话是不是讲了一半，或者这一段的逻辑还没结束，到点就切。
- 语义分块是让机器先读懂大概意思，通过计算前后句子的相似度，在话题发生转变的波谷处才进行切一刀。
- 为什么要优化？
    - 保证切出来的每一个 Chunk 都是一个语义完整的知识单元。
    - 完整的逻辑块更容易被向量检索匹配到，回答会更有条理。
- 参考实现思路与技术路线：

    a. Embedding 相似度计算：

        - 原理：使用 Embedding 模型将每个句子转化为向量。
        - 逻辑：计算 `句子 A` 和 `句子 B` 的余弦相似度。如果相似度很高（如 0.8），说明在讲同一件事；如果突然降到 0.3，说明话题变了，这里就是最佳切分点。

    b. 利用 LlamaIndex 现成组件：

        - 去查阅 `llama-index` 的官方文档，关键词是 `SemanticSplitterNodeParser`。

- 研究它的参数：`buffer_size`（计算范围）、`breakpoint_percentile_threshold`（切分阈值）。调整这些参数，找到最适合的切分粒度。

c. 可视化验证：

- 可以写一个小脚本，打印出切分结果。对比一下固定切分和语义切分出来的段落，看看后者是否更像人类的自然段落。

二：流式响应

- 目前前端是“发请求 -> 等待转圈 -> 显示全部文字”。
- 流式响应是“发请求 -> 第一个字出来 -> 一个个字蹦出来”。
- 为什么要优化？
    - 不让用户干等，提升用户体验。
- 参考实现思路与技术路线：

a. 后端协议选择 (SSE)：

- 使用 Server-Sent Events (SSE) 协议，比 WebSocket 更轻量，更适合单向文本流。
- 在 FastAPI 中，研究 `StreamingResponse` 类。
- LlamaIndex 的 Chat Engine 有一个 `stream_chat` (同步) 或 `astream_chat` (异步) 方法，它返回的是一个生成器 (Generator)。后端需要把这个生成器包装成 SSE 格式推给前端。

b. 前端数据流处理：

- 前端 `fetch` 请求不再是直接 `await response.json()`。
- 需要使用 `response.body.getReader()` 来读取流数据。
- 每读到一个数据块（Chunk），就把它拼接到当前的 `messages` 状态里，触发 React 重新渲染，从而产生打字机效果。

...

2：资料中心优化

一：持久化缓存机制

- 生成摘要和测验是非常消耗Token和时间的操作。

- 如果用户昨天生成过这本书的摘要，今天点开还要再等 20 秒重新生成，这就是极大的浪费。
- 为什么要优化？
    - 第二次访问时，直接可以得到。
    - 节省API调用成本。
- 参考实现思路与技术路线：

    a. Hashing：

        - 如何确定这份文件是否变过？不要只看文件名。
        - 建议对文件内容计算 MD5 或 SHA-256哈希值。以此作为唯一的 `cache_key`。

    b. 数据库设计：

        - 设计一张简单的 SQLite 表（例如 `document_cache`）。
        - 字段建议： `file_hash` (主键), `summary_text`, `quiz_json`。

    c. 拦截逻辑：

        - 在调用 LLM 之前加一层判断：
            - *Check*: 数据库里有这个 Hash 的记录吗？
            - *Hit*: 有 -> 直接返回数据库里的文本（耗时 0s）。
            - *Miss*: 没有 -> 调用 LLM 生成 -> 存入数据库 -> 返回结果。

二：并发生成加速

- 长文档摘要的原理是分治法：把 50 页书切成 10 块，每块先做小摘要，最后汇总。
- 可以把串行改为并行
- 为什么要优化？
    - 缩短等待时间。
- 参考实现思路与技术路线：

    a. 尽可能达到API 额度极限：

        - DeepSeek API 是支持并发调用的。
        - 研究 LlamaIndex `TreeSummarize` 类的参数。关注 `use_async=True` (是否异步) 和 `num_concurrent` (并发数量)。尝试调大数字，观测速度提升。

    b. 异步编程 ：

- 确保从读取文件、切分到请求 API 的整个链路都是 `async/await` 的，不要让某个同步操作阻塞了整个线程。

c. 压力测试与容错：

- 并发高了可能会触发 API 的 Rate Limit (速率限制) 报错。
- 如果并发太快报错了，怎么加一个简单的重试机制？

...

议程二：系统测试报告与结题答辩

为应对结题答辩，需完成以下交付物：

- **系统测试报告：**
- **结题答辩 PPT：**

三、 会议决议与行动项

1. 总体决议：

- **代码封板时间：2025年12月20日**。在此之后严禁进行大规模重构，仅允许修复Bug，以确保答辩演示的稳定性。

2. 任务分配与截止日期：

任务内容	负责人	截止日期
系统测试报告撰写	陈子涵	2025年12月28日
结题答辩	陈子涵	2025年12月28日

四、 会议总结

- **从技术角度看：** 拟定的四项优化（语义切分、流式响应、缓存、并发）将使本系统在技术复杂度上显著优于普通 CRUD 项目，是争取高分的关键筹码。
- **从进度角度看：** 时间紧迫。接下来的两周是“开发+测试”并行期，需严格遵守时间点，避免因临场改代码导致演示翻车。